

كلية الهندسة المعلوماتية - جامعة دمشق

مشروع مادة

المترجمات

المجموعة ١٨ — مترجم لجزء من Python/Flask و Jinja2/HTML و CSS

إشراف: د. باسم قصيبة

إعداد وتقديم الطلاب:

- فردوس محمد هديه
- محمد فهد جمعه
- حسن منذر يوسف
- احمد عبد المجيد الفروان
- شاها اسماعيل جاويش

أولاً: تعريف باللغة

ورد هذا السؤال بصيغة سؤال كمطلوب أساسي في مشروعنا: لغة Python هي لغة برمجة تعتمد على الإزاحة (Indentation) بدل الأقواس لتحديد الكتل البرمجية، وتستخدم على نطاق واسع لبناء تطبيقات الويب عبر إطار العمل Flask. أما Jinja2 فهي لغة قوالب تسمح بكتابة واجهات HTML مع حقن البيانات وعناصر التحكم مباشرة، وتكملها CSS لتزيين الواجهات.

في مشروعنا قمنا ببناء مترجم تعليمي لهذه اللغات (مجموعة فرعية منها) حيث قمنا بتحقيق مراحل المترجم الخمس كاملة، ويمكن باستخدام اللغة التي قمنا ببنائها القيام بما يلي:

- تعريف المتغيرات وإسناد القيم إليها لجميع الأنواع الأساسية: أعداد ونصوص وقيم منطقية وقوائم (List) وقواميس (Dict).
- تعريف التوابع function مع معاملاتها وبيان المسار مع الديكوريترات (Decorators) الخاصة بـ Flask مثل `app.route@` وبيان طرق HTTP.
- الوصول إلى عناصر القوائم والقواميس عبر الفهرس Index وأيضاً خصائص الكائنات (Attributes) كالوصول إلى `request.form`.
- استخدام الجمل الشرطية `if/elif/else` وتوابعهن المتسلسلة.
- استخدام أمر الإرجاع `return` واستيراد المكتبات بـ `import`.
- استدعاء التوابع مع الوسائط (Arguments) والوسائط المسماة (kwargs) مثل `render_template('products.html', products=products)`.
- كتابة عمليات حسابية ومنطقية ومقارنات بأوليياتها الصحيحة.
- كتابة واجهات HTML كاملة مع كتل Jinja2: التعابير `{% if %}` و `{% set %}` ودوال الفلتر مثل `upper|` و `for %}` و `{% }` و كتل التحكم `{% if %}` و `for %}`.

• كتابة أنماط CSS داخل وسوم `<style>` وأيضًا ملفات CSS مستقلة، وتوليد الأكواد النهائية منها في مسارات صحيحة داخل التطبيق المولد.

ما سبق إضافة إلى الكثير من التفاصيل الأخرى قمنا بتحقيقها في هذا المشروع، وسنتعرف على كيفية قيامنا بذلك في الأقسام التالية من التقرير.

ثانيًا: بناء قواعد اللغة اللفظية والنحوية

في البداية قمنا بكتابة قواعد اللغة اللفظية والنحوية باستخدام لغة قواعد g4 حيث قمنا بإنشاء ستة ملفات بلاحقة .g4، فلكل لغة من اللغات الثلاث (Python و Template و CSS) ملفان: ملف Lexer يحوي القواعد اللفظية، وملف Parser يحوي القواعد النحوية. والملفات هي:

```
PythonParser.g4 و PythonLexer.g4
TemplateParser.g4 و TemplateLexer.g4
CssParser.g4 و CssLexer.g4
```

حيث تم تجميع رموز كل لغة وكتابتها في الملف الخاص بال Lexer على شكل Tokens ثم تم إنشاء القواعد وكتابتها في الملف الخاص بال Parser، مع مراعاة ترتيب القواعد بحيث يتعرف أي رموز لها الأولوية في التحليل.

وبعد ذلك قمنا باستخدام أداة ANTLR لتوليد ال Lexer وال Parser بناءً على هذه القواعد وكلاسات Visitor و Listener المساعدة، إضافة إلى كلاساتنا الخاصة التي تكمل بناء الكومبايلر مثل كلاسات العقد التي سنبنى منها الشجرة AST وكلاسات ال Visitors الدلالية وكلاسات جدول الرموز والتشخيصات وغير ذلك من الكلاسات الضرورية المساعدة والمكملة، وهي منظمة في حزم (Packages) داخل مجلد src/main/java وهي: ast و parseTree و symbolTable و visitor و cli.

واجهتنا تحديات خاصة في هذا البناء:

أولاً — مشكلة INDENT/DEDENT في Python: بما أن Python تحدد الكتل بالإزاحة وليس بالأقواس، فإن ANTLR لا يفهم ذلك بنويًا. الحل: أنشأنا كلاس PythonLexerBase.java يرث من كلاس Lexer ويتتبع مستوى إزاحة الأسطر عبر مكس (Stack). عند زيادة الإزاحة يولد Token اصطناعي INDENT، وعند نقصها يولد DEDENT (ربما عدة)، ويتجاهل الإزاحة داخل الأقواس () / [] / { } عبر عداد openedBrackets لأنها غير ذات معنى هناك.

ثانيًا — تعارض الأنماط (Modes) في القوالب: قالب Jinja2 يحتوي في الوقت نفسه على نص HTML وتعبيرات { % } وكتل { % % } ومحتوى CSS داخل <style>، وأقواس CSS { } قد تتداخل مع جميع ذلك. الحل: استخدام Lexer Modes بخمسة أوضاع: الوضع الافتراضي للنص، وضع HTML داخل الوسم، وضع حين فتح <style>، وضع محتوى CSS، ووضع تعبيرات/كتل Jinja. وهذا يمنع أي التباس بين الأقواس المختلفة.

ثالثًا: توليد الشجرة AST

الكلاسات التي تم توليدها باستخدام ANTLR عند استخدامها تعطينا Parse tree، ولكننا نحتاج لتحويلها إلى AST خاصة بنا لنتمكن من استخدامها في المراحل التالية من الكومبايلر.

في البداية قمنا بإنشاء حزمة باسم ast، وتحتوي الـ AST على قواعد اللغة بالإضافة إلى الرموز الرئيسية فقط التي تعطي قيمة أو تحمل معنى منطقي فقط. وبما أن مشروعنا يترجم لغتين منفصلتين (Python للواجهة الخلفية، و Template للواجهات) فقد قمنا ببناء شجري AST منفصلتين:

Python AST • عقدها تعبر عن برنامج Python مثل ProgramNode و FunctionDefNode و ImportNode و ReturnNode و IfNode و AssignNode و DecoratorNode و BinaryExprNode و CallExprNode و DictNode و IndexExprNode و AttributeExprNode.

Template AST • عقدها تعبر عن قالب HTML/Jinja مثل HtmlElementNode و TextNode و JinjaSetNode و JinjaForNode و JinjaIfNode و JinjaExpressionNode و CssStyleNode.

أما السبب في شجرتين منفصلتين فأن Python لها تراكيب لا توجد عند HTML/Jinja (دوال وديكوريتر واستيراد) وبالعكس HTML/Jinja لهما عناصر وسمات لا توجد في Python، فبناء شجرة موحدة سيجعل العقد والمعالجات مختلطة ومعقدة.

أفضل طريقة لتمثيل كل قاعدة هي بإنشاء صف class خاص بها واعتبار القاعدة كائنًا Object، وبذلك تحوي كل حزمة AST على كلاسات وفي كل كلاس ما يعبر عن القاعدة من رموز وقيم ومؤشرات على كائنات من صفوف أخرى، مع بناء constructor.

وقد استفدنا من مفاهيم الـ OOP بشكل كبير في هذه البنية:

• **الوراثة:** كلاس ASTNode.java مجرد يحمل معلومات الموقع (اسم الملف file ورقم السطر line وموضع العمود column) عبر التابع setLocation، ويرث منه PythonNode و TemplateNode، ويرث من هذين كل العقد الفرعية (٢٧ عقدة).

• **التجريد (Abstraction):** العقدة الأب تحدد التابع accept المجرد، وكل عقدة تنفذه.

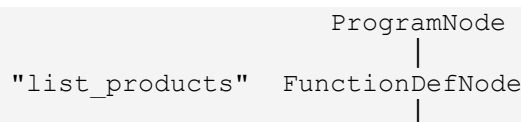
• **تعدد الأشكال (Polymorphism):** طبقنا نمط الـ Visitor. عرّفنا واجهة ASTVisitor تحوي عدداً كبيراً من التوابع المسماة visit، ودالة accept في كل عقدة تقوم بتسليم العقدة إلى الـ Visitor؛ فيختار الـ runtime تلقائياً التابع visit المناسب لنوع العقدة الفعلي دون أي تحويل يدوي.

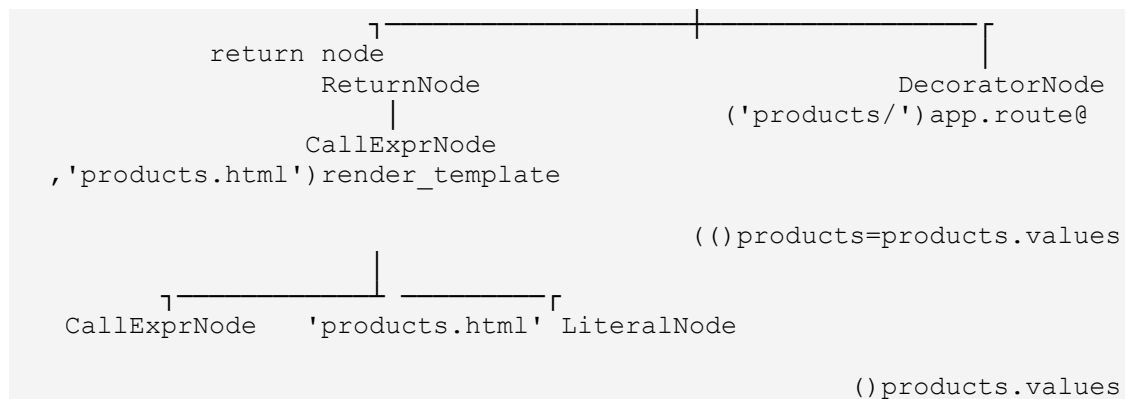
ونتيجة لذلك يمكن استخدام الشجرة نفسها مع ثلاثة زوّار مختلفين: PrintVisitor للطباعة، و SemanticAnalyzer للتحليل الدلالي، و ProjectGenerator لتوليد الكود. وكل عقدة مؤلفة من أبناء من نفس نوع العقد أو أصغر، فتمثل شجرة فعلية يسهل الطواف عليها عودياً.

ولتحويل الـ Parse Tree إلى الـ AST الخاصة بنا أنشأنا حزمة parseTree فيها صفان: PythonASTBuilder و TemplateASTBuilder، يرثان من كلاسات الـ Visitor المولدة من ANTLR، وفيهما نقوم بعملية @Override لتوابع الزيارة التي يتم من خلالها إنشاء عقد AST من كل Context نحصل عليه، مستخرجين القيم المهمة مثل أسماء التوابع والمعاملات والسمات، ومستدعين setLocation من الموقع الفعلي للتوكين في الملف.

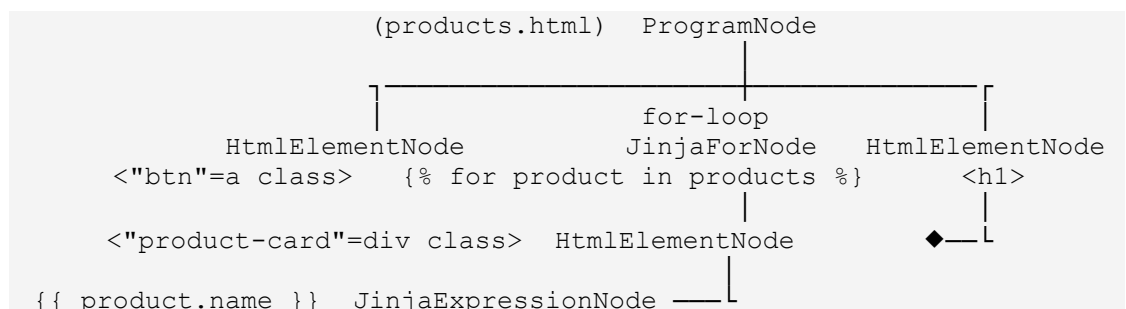
مخطط يوضح بنية الشجرتين

أولاً: Python AST — مثال: الدالة list_products من التطبيق التجريبي:

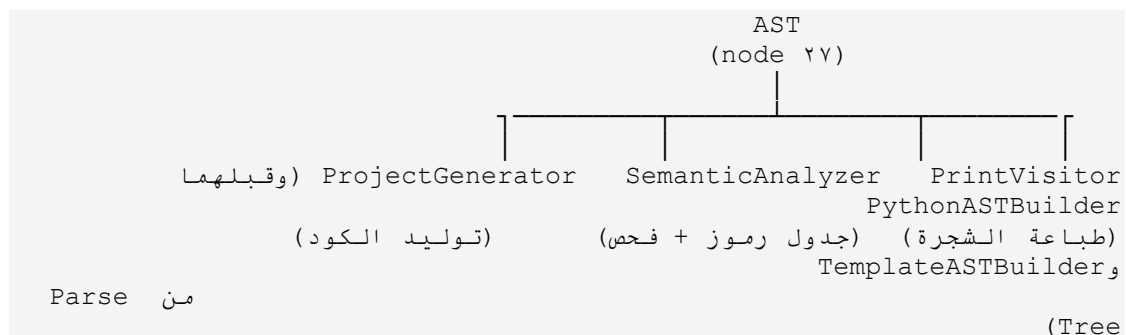




ثانياً: **Template AST** — مثال: قالب `products.html` التجريبي:



ثالثاً: رحلة الشجرة عبر المترجم — الشجرة نفسها تمر على ثلاثة زوّار يحوي كل منها `visit` لكل نوع عقدة:



وكل عقدة تحمل معها `file` و `line` و `column` فتبقى مواقع الأخطاء دقيقة في كل مرحلة، والمخطط المفصل بجميع العقد الـ ٢٧ موجود في وثيقة [مخطط AST](ast-diagram.md).

رابعاً: توليد جدول الرموز `SymbolTable`

سوف نعتبر جدول الرموز هو عبارة عن `Object` من كلاس `SymbolTable.java` يحتوي على `Scope` الحالي ومرجعاً إلى النطاق العالمي، وكل `Scope` هو عبارة عن `Object` من كلاس `Scope.java` مؤلف من اسم ونطاق أب ومجموعة من الرموز (`Symbols`) المعرفة فيه، وكل `Symbol` هو عبارة عن `Object` من كلاس `Symbol.java` مؤلف من اسم ونوع (`Kind`) يُحدد إن كان متغيراً أو دالة أو معاملاً أو استيراداً.

و `Object` الـ `SymbolTable` إضافة إلى ما سبق يحتوي على قواعد لإضافة العناصر والسكوبات والتحقق من وجود عنصر في سكوب ما أو قابلية الوصول إليه من ذلك السكوب، فعند تعريف أي متغير أو تابع أو رمز ما يتم إضافته إلى كائن `SymbolTable` الخاص بالبرنامج، وعند الدخول إلى جسم دالة أو شرط أو حلقة ينشأ

Scope جديد ويصبح الحالي، وعند الخروج يعود المحلل للنطاق الأب، وهكذا نحصل على نطاقات متداخلة (Nested Scopes).

ونقطة تُظهر قوة تصاميمنا: عند تحليل استدعاء `render_template('products.html')`، `products=products` نفتح في جدول الرموز Scope خاصاً باسم `template_products.html` ونعرّف متغيرات القالب فيه، وهذه هي الآلية التي تسمح للتحليل الدلالي بالتحقق من متغيرات Jinja داخل كل قالب على حدة، وستظهر هذه النطاقات الخاصة بوضوح في طباعة جدول الرموز كما نعرضها في قسم الطباعة.

خامساً: معالجة الأخطاء Error Handling

احتجنا إلى عدة كلاسات مكملية لجدول الرموز لتحقيق معالجة الأخطاء في مرحلتين:

المرحلة الأولى — الأخطاء النحوية (Syntax Errors): أنشأنا كلاس `BailErrorListener.java` مرتبط بال `Parser`، وعند حدوث خطأ نحوي في أي ملف نوقف التحليل فوراً عند أول خطأ، نسجل موقعه (الملف والسطر والعمود)، ونطبع رسالة على `stderr` وننهي الـ CLI برمز خروج غير صفري، وبالتالي لا نصل أبداً إلى مرحلة التوليد بكود ناقص.

المرحلة الثانية — الأخطاء الدلالية (Semantic Errors): أداة ANTLR تولد معالجة الأخطاء الأساسية بناءً على القواعد التي كتبناها، ولكن يوجد الكثير من الأخطاء الإضافية التي من الصعب جداً التعبير عنها باستخدام القواعد وحدها وقد تؤدي إلى إضافة عدد كبير من القواعد لمجرد تحقيق التقاط نوع واحد من الأخطاء، الأمر الذي يمكن تحقيقه بسطور بسيطة من الكود خلال عملية زيارة العقد في `SemanticAnalyzer.java`. وهذه هي الفائدة الأكبر من معالجة الأخطاء بهذه الطريقة حيث تعطي مرونة عالية للغة وسهولة في التقاط الحالات الخاصة والـ Edge Cases.

ويجمع كلاس `DiagnosticCollector.java` الأخطاء مع معلومات الموقع كاملة، ويعرضها كلاس `CompilerError.java` بالتنسيق المطلوب. وبعد نهاية التحليل إن وجدت أخطاء نمنع التوليد ونطبعها. وسنورد هنا حالات الأخطاء التي تمت معالجتها، وهي ١٢ خطأ توزعت على اللغتين:

أخطاء Python (٥):

- عند تعريف دالة أو معامل مكرر في نفس النطاق (`PY_DUPLICATE_SYMBOL`).
- عند ورود `return` خارج جسم دالة (`PY_RETURN_OUTSIDE_FUNCTION`).
- عند تكرار نفس مسار Flask مع نفس طرق HTTP (`PY_DUPLICATE_ROUTE`).
- عند استدعاء `render_template` باسم قالب غير موجود (`PY_RENDER_MISSING_TEMPLATE`).
- عند دمج نوعين غير متوافقين مثل عدد مع نص (`PY_TYPE_MISMATCH`).

أخطاء Template (٧):

- عند اختيار المعرف `id` نفسه لعنصرين (`TPL_DUPLICATE_ID`).
- عند ورود رابط `a` بلا خاصية `href` (`TPL_MISSING_HREF`).
- عند ورود عنصر صورة أو سكربت بلا `src` (`TPL_MISSING_SRC`).
- عند ورود وسوم `<style>` فارغة (`TPL_EMPTY_STYLE`).
- عند استخدام فلتر Jinja غير مدعوم (`TPL_UNSUPPORTED_FILTER`).
- عند استخدام متغير غير معرف في تعبير أو كتلة Jinja (`TPL_UNDEFINED_VARIABLE`).
- عند التكرار على متغير غير معرف في كتلة `{% for %}` (`TPL_UNDEFINED_ITERABLE`).

ولتحقيق الربط الدلالي بين الشجرتين، يحلل المحلل ملف Python أولاً بعد تسجيل أسماء القوالب الموجودة فعلياً، فيمكن اكتشاف القالب المفقود، ثم عند العثور على `render_template` تُجمع أسماء `kwargs` حسب اسم القالب في خريطة `templateContexts`، وعند تحليل كل قالب يفتح `Scope` خاصاً ويعرّف متغيراته فتفحص كتل `Jinja` المتغيرات المتاحة فعلياً.

سادساً: توليد الكود Code Generation

يتم توليد الكود باللغة الهدف عن طريق كلاس `ProjectGenerator.java` الذي ينفذ واجهة `ASTVisitor` ويعمل على الشجرتين معاً، فمن `Python AST` يولّد ملف `app.py` الخاص بـ `Flask`، ومن نفس الـ `Visitor` على `Template AST` يولّد ملفات القوالب `templates/*.html`، ويولّد أيضاً ملف `requirements.txt` لتشغيل التطبيق. والملفات المولدة مترابطة، فالمسارات في `app.py` هي نفس الأسماء التي تستدعيها `render_template` والروابط الموجودة في القوالب، وهذا الترابط هو الذي ينتج تطبيقاً يعمل فعلاً عند تشغيله.

ولا تُنسخ قوالب `HTML` كما هي: القالب يُفكك ويحلل إلى `Template AST` ثم يعيد `ProjectGenerator` توليده من العقد بكتابته من جديد. أما ملفات `CSS` المستقلة فبعد اجتياز قواعد `CssParser` يُحفظ نصها مباشرة تحت `static/css`، لأنها ليست من الشجرتين المطلوبتين صراحة.

وكما في أي مترجم، فقد اعتمدنا قواعد تقابل (`Mapping Rules`) بين الشجرة والنص المولد، وسنورد هنا بعض الأمثلة على الاستثناس والفهم (وليس الحصر):

- عقدة `DecoratorNode` الخاصة بـ `app.route@('products/')` تولّد السطر `app.route@('products/')` قبل تعريف الدالة مباشرة.
- عقدة `FunctionDefNode` للدالة `def add_product()` : مع معاملاتها تولّد `def add_product()` : ثم جسم الدالة من عقد الأبناء بشكل عودي مع إزاحة صحيحة لكل مستوى.
- عقدة `CallExprNode` للاستدعاء `render_template('products.html', products=products.values)` تولّد الاستدعاء نفسه بوسائطه المسماة.
- عقدة `HtmlElementNode` للعنصر `<div class="product-card">` تولّد وسم الفتح والسمات ثم تعيد توليد أبنائها ثم وسم الإغلاق.
- عقدة `JinjaForNode` للكتلة `{% for product in products %}` تولّد سطر الفتح وتعيد توليد جسمها عودياً ثم `{% endfor %}`.
- عقدة `CssStyleNode` تعيد كتابة محتوى `CSS` داخل وسم `<style>` كما هي.

وبما أن المولد يستخدم نفس التابع `visit` لكل نوع عقدة وفق نمط الـ `Visitor`، فإن الأكواد تبقى متسقة ومقروءة، ويخضع الناتج في الأخير لفحص `python -m py_compile` ليتم التأكد من سلامته ثم تُشغّل عليه الاختبارات.

سابعاً: الطباعة والواجهات (Printing and the Web Interfaces)

الطباعة: تحققنا المتطلب بنوعين من التوابع المقروءة:

- طباعة الشجرة: `PrintVisitor` يمر على كل عقدة وأبنائها بشكل مقروء، فلكل عقدة تابع `visit` يطبعها مع معلومات موقعها (الملف والسطر والعمود) ومع مسافة بادئة يتزايد عمقها حسب عمق العقدة في الشجرة، فيظهر الشكل النهائي هرمياً مفهوماً.

• **طباعة جدول الرموز:** `SymbolTable.printSymbolTable()` يمر على النطاقات من الأب إلى الأبناء ويعرض كل رمز في نطاقه مع نوعه، وتظهر فيه نطاقات القوالب مثل `template_products.html` وبجوارها متغيراتها، وهذا دليل مرئي على الربط الدلالي بين الشجرتين.

الواجهات: التطبيق المولد تطبيق ويب لإدارة المنتجات بنمط CRUD كامل، بمسارات خمسة مترابطة بتنقل سلس:

الوظيفة	Route	Method
قائمة المنتجات	/products	GET
نموذج الإضافة	/products/add	GET
حفظ المنتج	/products/add	POST
تفاصيل المنتج	/products/<int:id>	GET
حذف المنتج	/products/<int:id>/delete	POST

وقد استخدمنا لخزن المنتجات قاموساً وعدداً مستقلاً `counters["next_id"]`، فهذا يمنع تكرار المعرفات أو أخطاء الفهرسة بعد الحذف. وعند انكسار شرط الإدخال تعالج الواجهات الحالات: الحقول الإلزامية الفارغة تعيد `abort(٤٠٠)`، والمعرف غير الموجود يعيد `abort(٤٠٤)`. والتنقل بين الواجهات يتم عبر `redirect(url_for(...))` وبالدوال المسماة فلا تنكسر الروابط عند تغيير المسارات.

ثامناً: استخدام كل ما سبق في البرنامج النهائي

في النهاية قمنا بتعريف كلاس `CompilerCli` والتابع `main` وهو نقطة دخول الكومبايلر، حيث يُحدد فيه مسار ملفات البرنامج المراد عمل `compile` لها، ويدعم الخيارات:

```
python--<file>: ملف Python المراد ترجمته.
templates--<directory>: مجلد القوالب HTML/Jinja و CSS.
output--<directory>: مجلد إخراج التطبيق المولد.
print-ast--: طباعة شجري AST.
print-symbols--: طباعة جدول الرموز.
diagnostics--: طباعة التشخيصات.
```

ومن أجل الملفات المحددة يتم قراءة الملفات وتميرها في الـ `Lexer` والـ `Parser` واحداً واحداً، مع مرور الأخطاء النحوية عبر `BailErrorListener` وبناء الشجرتين وملء `SymbolTable` و `templateContexts`، فيتم في النهاية إظهار كل الأخطاء النحوية والدلالية للمستخدم إن وجدت، وإن لم توجد يتم توليد التطبيق الكامل في المجلد المحدد.

ولإثبات أن الأجزاء المولدة تعمل معاً قمنا ببناء آلية اختبار كاملة:

- **٢٦ اختبار JUnit** على الجافا تغطي بناء الـ `AST` و `Template` و `CSS` والتحليل الدلالي والطباعة والتوليد.
- **٨٠ اختبارات قبول Python (pytest)** تستورد التطبيق المولد نفسه وتجرب دورة `CRUD` كاملة واستجابات `٤٠٤/٤٠٠` والقالب المفقود والـ `CSS` الصالح والخاطئ، مع تطبيقات تجريبية `test1/test2/test3`.
- **فحص python -m py_compile** على `app.py` المولد.

وقد شُغل البناء النظيف من الصفر وكانت النتيجة النهائية: JUnit ٢٦/٢٦ و ٨/٨ pytest و py_compile ناجح، والناتج يعمل في المتصفح على الرابط <http://127.0.0.1:5000/products>.

النهاية

هذا هو النموذج: مشروع يحقق سلسلة المترجم كاملة ومتراصة — من القواعد اللفظية والنحوية للغات الثلاث، إلى شجري AST بنمط Visitor متعدد الاستخدامات، وجدول رموز بنطاقات متداخلة، واثنى عشر خطأ دلاليًا يربط الشجرتين، وتوليداً لتطبيق Flask يعمل ويخضع لاختبارات آلية حقيقية، وفق متطلبات المادة كاملة.